

CS336笔记 强化学习

1. 引言

 **本文主题：**实现 Zero/Few-shot **GRPO**（组相对策略优化，Group Relative Policy Optimization）算法和 **CoT**（思维链，Chain-of-Thought）。

预训练使用广泛的数据，令模型具有更泛化的能力；后训练 RL 中，希望模型在特定任务上达到高准确率，例如解决数学问题。这个更窄的目标至少在**两个方面**与预训练不同：

- **(a) 数据量：**我们不再拥有那么多数据；
- **(b) 训练目标：**从"覆盖率"（拥有广泛的知识库）转变为"**精确度**"（生成准确的回答）。

 **RL 的局限性：**高度依赖训练使用的模型和数据集，迁移效果未必好；速度慢且不稳定，不同随机种子下方差很大，微小实现细节的影响也很大。

RL 使用**问题数据集**提供训练样本，配合**评分函数**判断回答是否正确解决了问题。

GRPO 的目标函数如下——优化给定分布为 ρ 的 prompt 数据集 x 下的策略 π_θ ，使得各组内 response y 的平均评分/奖励 reward 尽可能高：

$$J_\theta = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)} [r(y|x)]$$

高层流程：**从模型中采样回答** → 用评分函数打分 → **强化正确的回答**。模型研究发现，使用思维链推理提示时，在强大的基础模型上进行 RL 可以显著提升其推理能力。

首先从数据集中采样数学问题 $x \sim \rho$ ，然后持续采样 token 直到达到回答末尾，记为 $y \sim \pi_\theta(y|x)$ 。上述目标导出了优化问题：

$$\theta^* = \arg \max_{\theta} J_\theta$$

2. 将语言模型视为策略

2.1 策略与动作的对应关系

在传统的强化学习中，我们考虑一个**策略**，它接收状态 s_t 并输出一个动作 a_t 。这个动作会产生一些奖励 $r_t = r(s_t, a_t)$ ，随后从某个下一个状态分布中采样得到后续状态 $s_{t+1} \sim \text{next state}(s_t, a_t)$ 。强化学习涉及**优化策略以获得高奖励**。



核心对应关系：参数为 θ 的因果语言模型 π_θ 定义了给定当前文本前缀 $y_{<t} = (y_1, y_2, \dots, y_{t-1})$ 时下一个 token y_t 的概率分布。在 RL 背景下：

- 下一个 token $y_t \rightarrow$ **动作** a_t
- 当前文本前缀 $y_{<t} \rightarrow$ **状态** s_t

因此，LM 是一个**分类随机策略**。

$$a_t \sim \pi_\theta(\cdot | s_t), \quad \pi_\theta(a_t | s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}$$

在使用策略梯度优化策略时，我们需要**两个基本操作**：

1. **从策略中采样：**从上述分类分布中抽取一个动作 a_t ；
2. **计算动作的对数似然：**评估 $\log \pi_\theta(a_t | s_t)$ 。

通常， s_t 是到目前为止已生成的部分补全/解决方案，每个 a_t 是解决方案的下一个 token；当发出结束文本 token（如 `<|end_of_text|>`，或对于 `r1_zero` 提示则是 `</answer>`）时，回合结束。

2.2 轨迹

在强化学习中，我们从起始分布 $s_0 \sim \rho$ 中采样的初始状态开始，然后从策略 π_θ 中采样动作 a_t ，根据下一个状态分布 $s_{t+1} \sim \text{next state}(s_t, a_t)$ 转移，并重复此过程。这一系列状态和动作形成一条**（有限视野）轨迹**：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$$

其中 T 是轨迹的长度，即 a_T 是结束文本 token，或者达到了最大 token 生成预算。

在我们的场景中， $s_0 \sim \rho$ 是 prompt x ，包含要解决的数学问题（可能应用了 CoT 提示或少样本示例）。从该前缀开始逐个采样 token，下一个状态是旧前缀与生成 token 的连接，即 $s_{t+1} = (s_t, a_t)$ 。轨迹也称为**回合（episodes）**或**展开（rollouts）**。

2.3 奖励与回报



该场景下，**不衡量**轨迹中每步生成的回报 $r_t |_{t < T}$ ，只看**最终答案是否正确** r_T （取值为 **0 或 1**）。

3. 策略梯度

3.1 基本推导

为了最大化评分函数，使用**梯度上升**（注意是最大化评分而非最小化损失，所以是梯度上升而非梯度下降）：

$$J_{\theta} = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y|x)]$$

模型的参数更新逻辑为：

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J_{\theta_k}$$

因此，需要能够从样本中估计梯度 $\nabla_{\theta} J_{\theta_k}$ 。

梯度重写推导如下（对数导数技巧 / log-derivative trick）：

$$\begin{aligned} \nabla_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y|x)] &= \sum_y \nabla_{\theta} \pi_{\theta}(y|x) r(y|x) \\ &= \sum_y \pi_{\theta}(y|x) \nabla_{\theta} \log \pi_{\theta}(y|x) r(y|x) \\ &= \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y|x) \nabla_{\theta} \log \pi_{\theta}(y|x)]. \end{aligned}$$



推导要点：第一步将期望展开为概率表达式，把求导挪到里面；第二步利用 $\nabla f(x) = f(x) \cdot \nabla \log f(x)$ ；第三步利用乘法交换律，将 $r(y|x)$ 提前。

进一步地，**参数更新（梯度贡献）逻辑为：**

$$\theta \leftarrow \theta + \alpha \cdot r(y|x) \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

正样本的 $r(y|x)$ 为 **1**，负样本的 $r(y|x)$ 为 **0**，所以**只有正样本**能够按照其对数概率的比例影响参数更新，负样本根本没有梯度贡献。

对于正样本 $r(y|x) = 1$ ，参数更新是使：

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(y|x)$$

梯度 $\nabla_{\theta} \log \pi_{\theta}(y|x)$ 是 $\log \pi_{\theta}(y|x)$ 数值上升最快的方向。因此，**模型会朝着提高正样本对数概率的方向更新**。此为基本的强化学习训练循环。

3.2 算法实践

对于给定的一组（假设 B 条 prompt）样本 x ，每个 prompt 对应生成 G 个 response y ，**策略梯度估计**如下：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G r(y^{(i,j)} | x^{(i)}) \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)})$$

3.3 基线

由于

$$\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(y|x)] = \int \pi_{\theta} \nabla_{\theta} \log \pi_{\theta} dy = \int \nabla_{\theta} \pi_{\theta} dy = \nabla_{\theta} \int \pi_{\theta} dy = \nabla_{\theta} [1] = 0,$$

策略梯度的下述变形成立，只要 b 是常数或者**不依赖于** y 的函数：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)} [(r(y|x) - b) \nabla_\theta \log \pi_\theta(y|x)] = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)].$$

基线可以**调节策略梯度估计器的方差**。估计器写为样本均值 $\frac{1}{n} \sum_{i=1}^n Z_i$ ，其中

$$Z_i = (r(y|x) - b) \nabla_\theta \log \pi_\theta(y|x), \text{ 方差为 } \frac{\mathbb{E}[Z_i^2] - \mathbb{E}[Z_i]^2}{n}。$$

使用基线方法之后，例如 $b = 0.5$ ，则**正负样本均会有梯度贡献**。

3.3.1 GRPO 算法中的基线应用

在 GRPO 中应用**组均值基线**（ b 为该组所有 response 评分的均值）：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)})$$

其中**组均值**为：

$$\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} | x^{(i)})$$

尽管 μ_i 依赖于 y ，但其在 $\frac{G-1}{G}$ 的缩放因子下**保持了期望**：

$$\begin{aligned} & \mathbb{E}_{y^{(1)}, \dots, y^{(G)} \sim \pi_\theta(y|x)} \left[\frac{1}{G} \sum_{j=1}^G (r(y^{(j)} | x) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(j)} | x) \right] \\ &= \frac{1}{G} \sum_{j=1}^G \mathbb{E}_{y^{(2)}, \dots, y^{(G)} \sim \pi_\theta(y|x)} \left[\mathbb{E}_{y^{(1)} \sim \pi_\theta(y|x)} \left[(r(y^{(1)} | x) - \mu_i) \nabla_\theta \log \pi_\theta(y^{(1)} | x) \right] \right] \\ &= \frac{1}{G} \sum_{j=1}^G \mathbb{E}_{y^{(1)} \sim \pi_\theta(y|x)} \left[\left(r(y^{(1)} | x) - \frac{1}{G} r(y^{(1)} | x) \right) \nabla_\theta \log \pi_\theta(y^{(1)} | x) \right] \\ &= \frac{G-1}{G} \mathbb{E}_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \end{aligned}$$

进一步，**除以标准差**如下，此时**不再保持估计器的期望**：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \log \pi_\theta(y^{(i,j)} | x^{(i)})$$

更进一步，添加**序列长度归一化项**如下，又会改变期望：长序列中的 token 相对于短序列中的 token **被降权**，而非所有序列中的 token 被平等对待。

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_\theta \left(\frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_\theta(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right)$$

4. On-policy V.S. Off-policy



On-policy (同步 RL)

给定 x : 推理采样生成 $y \sim \pi_{\theta}(y|x)$
→ 评分 (得到 $r(y|x)$ 、优势 A) → 计算策略梯度 → 更新模型参数。

每轮迭代都重新采样。



Off-policy (异步 RL)

每次迭代有 K 轮, 但只有第 1 轮采样生成 y , 第 2~ K 轮沿用旧的 y 和旧的优势 A , 但根据新模型计算 $\pi_{\theta}(y|x)$ 。

一批数据复用 K 次。

第 2~ K 轮沿用 (全部 G 个) 第一轮生成的优势 A 和 response y , 正样本 y 的 A 为正, 负样本 y 的 A 为负, 所以鼓励新模型朝正样本概率更高、负样本概率更低的方向演进。

Off-policy 的迭代流程:

1. 用【当前模型 v1】采样 $B \times G$ 个回答;
2. 计算奖励和优势值;
3. 计算梯度, 更新模型 → 得到【新模型 v2】;
4. **不扔掉数据**, 继续用同一批数据;
5. 再次计算梯度 (用 v2 算 `log_prob`), 更新 → v3;
6. **还不扔掉**, 继续用同一批数据;
7. 第三次计算梯度 (用 v3 算 `log_prob`), 更新 → v4;
8. 直到重复了 K 次, 才扔掉这批数据。

在 Off-Policy 的 K 轮更新中, 所有 G 个旧样本及其旧优势值 A 全程复用。第 2~ K 轮的梯度更新, 通过比较"新模型概率"和"旧模型概率", 自动实现了:

- 对旧优势为正的样本 (好回答), 若概率上升则**强化**, 若概率下降则**拉回**;
- 对旧优势为负的样本 (坏回答), 若概率上升则**惩罚**, 若概率下降则**减轻惩罚**。

Off-policy 中利用**重要性权重** $\frac{\pi_{\text{new}}}{\pi_{\text{old}}}$ 进行修正, 如果模型 v2 生成正样本的概率高于模型 v1, 则放大这种效果。

4.1 重要性重加权

异步 RL 中推理模型在不断 rollout 样本放入 replay buffer, 模型在不断更新权重 (全局只保留一份权重), 训练模型采样到该样本时, 其权重 π_{θ} 有可能已不是推理生成时的权重 π_0 。

根据以下公式 (把求期望的积分公式展开即可得到), 对异步 RL 策略梯度校准, **确保期望不发生变化**:

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right] = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]$$

重加权后的估计器虽然**期望零偏差**，但**方差较高**，因为序列级重要性权重是各 token 权重的连乘：

$$\frac{\pi_\theta(y|x)}{\pi_0(y|x)} = \prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}$$

为了降低估计器方差，采用 **token 级重加权估计器**：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} \left[r(y|x) \sum_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]$$

4.2 token 级重加权

令 $\tilde{\pi}_t$ 表示**替代策略**，除步骤 t 从 π_θ 采样外，其他时间步都从旧策略 π_0 采样：

$$\tilde{\pi}_t(y|x) = \left(\prod_{s=1}^{t-1} \pi_0(y_s | x, y_{<s}) \right) \pi_\theta(y_t | x, y_{<t}) \left(\prod_{s=t+1}^{\text{len}(y)} \pi_0(y_s | x, y_{<s}) \right)$$

假设回答都具有相同的长度 L ，token 级重加权优化的是每个 $\tilde{\pi}_t$ 下的**期望奖励**：

$$J_{\text{token}}^\theta = \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right]$$

策略梯度推导如下（最后一步遵循对数导数技巧）：

$$\begin{aligned} \nabla_\theta J_{\text{token}}^\theta &= \nabla_\theta \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right] \\ &= \nabla_\theta \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \right] \right] \\ &= \mathbb{E}_x \left[\mathbb{E}_{y \sim \pi_0(y|x)} \left[\sum_{t=1}^L \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right] \right]. \end{aligned}$$

 **偏差说明**：token 级重加权假设时间步的 token y_t ，其前缀 $y_{<t}$ 和后缀 $y_{>t}$ 都是从 π_0 而非 π_θ 采样，因此**引发偏差**（ π_θ 离 π_0 越远，偏差越大）。这种偏差可以通过序列级重加权（重要性权重累乘方式）纠正，代价是方差增加。

4.3 重要性权重裁剪

token 级重加权 + 标准 GRPO 的损失函数如下，进行微分可得上述 token 重加权策略梯度估计器，但带有 **GRPO 优势和序列归一化**：

$$J_{\theta}^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}$$

令 $A_{(i,j)} = \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i}$ 表示回答 j 在提示 i 上的**优势值**，令 $w_t^{(i,j)} = \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}$ 表示第 t 个**重要性重加权项**，则裁剪后的目标函数为：

$$J_{\theta}^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min \left(A_{(i,j)} w_t^{(i,j)}, A_{(i,j)} \text{clip}(w_t^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon]) \right)$$

相应**策略梯度**为：

$$\nabla_{\theta} J^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \text{mask}_t^{(i,j)} w_t^{(i,j)} A_{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t})$$

其中 **mask** 函数为：

$$\text{mask}_t^{(i,j)} = \begin{cases} 1\{w_t^{(i,j)} < 1 + \varepsilon\} & \text{if } A_{(i,j)} \geq 0 \\ 1\{w_t^{(i,j)} > 1 - \varepsilon\} & \text{if } A_{(i,j)} < 0 \end{cases}$$

对重要性权重做裁剪，目的是**防止当前策略偏离旧策略太远**，也可以降低估计器方差。如果不担心离旧策略太远而希望更激进的更新，更简单的方法是**只做上限裁剪**：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \min(w_t^{(i,j)}, 1 + \varepsilon) A_{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t})$$

4.3.1 GSPO：序列级重加权的开 $1/L$ 次幂

token 级重加权有偏，GSPO 作者发现 GRPO 不稳定，采用以下损失函数。定义**序列级几何平均重要性权重**：

$$s^{(i,j)} = \left(\prod_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \right)^{\frac{1}{\text{len}(y^{(i,j)})}}$$

GSPO 裁剪目标函数：

$$J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \min \left(A_{(i,j)} s^{(i,j)}, A_{(i,j)} \text{clip}(s^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon]) \right)$$

策略梯度推导如下：

$$\begin{aligned}\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-gspo}} &= \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A_{(i,j)} \nabla_{\theta} s^{(i,j)} \\ &= \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G A_{(i,j)} s^{(i,j)} \left(\frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}) \right).\end{aligned}$$

5. GRPO 算法实践

 **工程架构：**需要推理引擎 **vLLM** 做生成，但 vLLM 工程上不支持计算梯度、反向传播；所以还需要额外的训练权重 **HuggingFace 模型**。两者通过权重同步协作。

5.1 On-policy 计算流程

代码块

- ```
1 第 N 轮迭代
2 1. 【同步】把 HF 模型的权重复制到 vLLM (sync_policy_weights)
3 2. 【采样】vLLM 生成一批文本 y (没有梯度)
4 3. 【重算概率】HF 模型对 (x, y) 做前向传播, 计算 log_probs
5 (这里产生了梯度, 但还没更新)
6 4. 【评分】CPU 计算奖励 r 和优势 A
7 5. 【更新】HF 模型计算 GRPO 损失, 调用 .backward() 更新权重
8 6. 【循环】回到第 1 步, 把更新后的 HF 权重再次同步给 vLLM
```

### 5.2 Off-policy 计算流程

代码块

- ```
1 第 N 次采样批次 (只采样 1 次, 但更新 K 次)
2  【第 0 步】把 HF 模型 v1 的权重复制到 vLLM
3  【第 1 步】vLLM 生成一批文本 y (冻结, 不再重新采样)
4  ===== 进入 K 轮内部更新循环 =====
5  【第 2 步】HF 模型 v1 对 (x, y) 做前向, 计算 log_prob_old
6  【第 3 步】CPU 计算奖励 r 和优势 A (基于 v1 的输出)
7
8  |  第 1 次梯度更新 |
9  | HF v1 计算损失, 反向传播 → 得到 HF v2 |
10 |
11 |
12 |  第 2 次梯度更新 (Off-Policy 关键) |
13 | - 使用旧数据 (x, y), 但现在是 HF v2 |
```



```

14 | - 计算 log_prob_new (用 v2 的概率) |
15 | - 结合旧 A 和重要性权重计算损失 |
16 | - 反向传播 → 得到 HF v3 |
17 |_____|
18 ... (重复 K 次)
19 ===== 退出内部更新循环 =====
20 【最后一步】把最终更新后的 HF 权重 (v_{K+1}) 同步给 vLLM
21 【下一轮采样】用更新后的 vLLM 采样全新的批次

```

5.3 最大化奖励 → 最小化损失

PyTorch 优化器中执行**梯度下降**，所以要把策略梯度转化为损失函数的形式。

前述 GRPO 的策略梯度为：

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{L_i} A_i \cdot \nabla_{\theta} \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})$$

需要构造标量损失 $L(\theta)$ 使 $\nabla_{\theta} L = -\hat{g}$ 。取

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{L_i} A_i \cdot \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})$$

就能成立。

5.4 变种：GRPO 算法消融实验

5.4.1 Dr. GRPO

标准差优势归一化和序列长度归一化导致策略梯度期望不再"正确"。

消融：移除标准差归一化 & 不按序列长度归一化：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} \left(r(y^{(i,j)} \mid x^{(i)}) - \mu_i \right) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)})$$

5.4.2 RFT

去掉组内评分均值基线：

$$\hat{g} \leftarrow \nabla_{\theta} \left[\frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G 1\{r(y^{(i,j)} \mid x^{(i)}) = 1\} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}) \right]$$

 **训练加速技巧：**优势为零的序列不需要传入模型（梯度贡献为零）。奖励为二元时，两种情况出现零优势：

- `baseline = "mean"`：组内所有序列奖励相同时
- `baseline = "none"`：任何奖励为零的序列

修剪零优势序列后可按比例减少 `gradient_accumulation_steps`。此优化对 **RFT 尤其有用**（零奖励序列多），但需确保修剪后梯度与未修剪版本一致。

6. PPO

6.1 算法流程

PPO 涉及四个模型：

模型名称	作用	是否可训练	类比
Actor （策略模型）	实际生成回答，被优化的对象	✅ 可训练	就是 GRPO 里的 π_θ （正在考试的考生）
Critic （价值网络）	预测"当前状态未来能拿多少总收益"	✅ 可训练	GRPO 没有，这是全新概念
Reference （参考模型）	计算 KL 散度，拉住 Actor 防止学歪	❌ 冻结	和 GRPO 里的 Ref 模型一样
Reward （奖励模型）	给最终的回答打分（数学题是规则，通用任务用训好的模型）	❌ 冻结	和 GRPO 里的 $r(y x)$ 类似

6.1.1 第 1 步：采样（Rollout）—— 用 Actor 生成一批数据

- 用当前 Actor 策略（ π_{old} ）采样一批 Prompt，生成对应的回答 y ；
- 关键点：同时记录每个 Token 的 log 概率（`log_probs_old`），用作后续"旧策略"的锚点。

6.1.2 第 2 步：评分（Scoring）—— 计算每个 Token 的"优势"（Advantage）



这是 PPO 最核心、与 GRPO 差异最大的地方。

- 奖励模型打分**：对每个生成的回答，奖励模型给出最终得分 r （数学题里就是 0 或 1）；
- Critic 预测价值**：在生成回答的每一个时间步 t ，Critic 网络输出标量 $V(s_t)$ ，预测"从这个前缀出发最终能拿多少分"；

3. 计算 GAE（广义优势估计）：

- 直观理解："奖励模型给的最终得分 r ，比 Critic 预测的 $V(s_t)$ 高多少？"
- 优势 $A = \text{实际拿到的分} - \text{预期拿到的分}$ ；
- $A > 0 \rightarrow$ 超出预期，应被**强化**； $A < 0 \rightarrow$ 低于预期，应被**抑制**。

6.1.3 第 3 步：计算损失 —— 三项目标一起优化

PPO 的损失函数由**三项**组成（GRPO 通常只有策略损失 + 可选 KL）：

 策略损失

$$L_{\text{clip}} = \min(\text{ratio} \cdot A, \text{clip}(\text{ratio}) \cdot A)$$

让模型多学"好行为"，ratio 过大时停止奖励防止学过头。

 价值损失

让 Critic 预测的 $V(s_t)$ 尽量靠近实际奖励 r 。

本质是回归任务，让评论家打分越来越准。

 熵奖励

鼓励模型保持探索多样性。

防止过早崩溃成只会输出固定答案的"复读机"。

其中 $\text{ratio} = \pi_{\text{new}} / \pi_{\text{old}}$ ，Clip 把 ratio 限制在 $[0.8, 1.2]$ 之间。

6.1.4 第 4 步：更新模型（梯度下降）

- 计算总损失 $L = L_{\text{policy}} + c_1 \cdot L_{\text{value}} - c_2 \cdot L_{\text{entropy}}$ ；
- 执行 `loss.backward()` 和 `optimizer.step()` ，同时更新 **Actor 和 Critic** 网络。

6.1.5 第 5 步：循环

- 用更新后的 Actor 重新采样，进入下一轮迭代。

6.2 和 GRPO 对比

维度	PPO	GRPO
优势来源	Critic 网络（额外模型）全局预测基线	组内平均分（无额外模型）局部基线
显存占用	4 个模型（高）	3 个模型（低）
训练目标	策略损失 + 价值损失 + 熵奖励	策略损失 + （可选）KL 惩罚
适用场景	通用任务（对话、代码、摘要）	数学推理等可验证任务
稳定性	依赖 Critic 训练质量，可能不稳定	组内归一化，天然稳定

步骤	PPO	GRPO
1. Rollout	Actor 采样，记录 <code>log_probs_old</code>	同左
2. 计算优势 A	Critic + GAE 公式	组内归一化 $((r - \mu)/std)$
3. 损失函数	$\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}) \cdot A) + \text{Value Loss} + \text{Entropy}$	$-(A \cdot \log \pi)$ (无裁剪，同策略)
4. 更新	更新 Actor + Critic	只更新 Actor

(注：部分内容可能由 AI 生成)